

DS340 Final Report

Authors: Hongyao Shao, Penny Lin

Introduction

Emotions are an important part of human life, influencing communication, decision-making, and relationships. In this project, we aim to determine whether a model could accurately predict emotions based on spectrograms generated from audio clips. A spectrogram is a visual representation of the frequencies of a sound signal, depicting how the intensity of different frequencies changes over time. The challenge lies in accurately mapping these spectrograms to specific emotional states, which is complex due to the variability in human speech and expression.

This project is motivated by the increasing demand for emotion-aware systems in fields like human-computer interaction, mental health monitoring, and entertainment. These systems could enable more natural interactions with virtual assistants or help therapists assess patients' emotional well-being.

To address this, we used convolutional neural networks, which are effective at processing image-like data such as spectrograms. Our approach involved converting audio clips into spectrograms and training a CNN to classify them into emotion categories. This method leverages the ability of CNNs to detect patterns in visual data, making it well-suited for emotion recognition from audio.

Methodology

Dataset Collection

We used the Ryerson Audio-Visual Database of Emotional Speech and Song ([RAVDESS](#)), which contains both song and speech audio clips recorded by 24 professional actors (12 female and 12 male). Each recording is labeled with one of eight emotions: neutral, calm, happy, sad, angry, fearful, disgust, and surprised. The dataset includes 1012 song clips and 1440 speech clips, providing a total of around 2452 audio samples. This dataset provides high-quality emotional audio clips and is widely used in emotion recognition research.

In addition to the RAVDESS dataset, we also recorded some portions of DS340 lectures to serve as real-world audio samples for final model testing.

Data Processing

First, we combined the datasets of songs and speech clips, converted each clip into a spectrogram to create an image representation of the sound frequencies over time, and grouped them by their corresponding emotion labels. We used Python libraries such as librosa and matplotlib to load the .wav files and generate spectrograms, following a method adapted from a StackOverflow reference. After preliminary model testing, we observed significant differences between the song and speech spectrograms that negatively affected classification performance. As a result, we separated the spectrograms into two distinct datasets: one for song clips and one for speech clips. Next, we used TensorFlow's ImageDataGenerator to preprocess the data and set up training and validation splits. Each spectrogram was resized to 128×128 pixels to ensure uniform input size for the CNN models, and pixel values were normalized to the $[0, 1]$ range. We utilized `flow_from_directory` with a validation split of 20%, automatically partitioning the dataset into training (80%) and validation (20%) subsets. Categorical labels were assigned based on the emotion classes associated with each spectrogram image. Instead of using a fixed hold-out test set from the original dataset, we evaluated model performance on the validation set during training. For real-world testing, we used additional lecture recordings collected independently to assess the model's generalization ability.

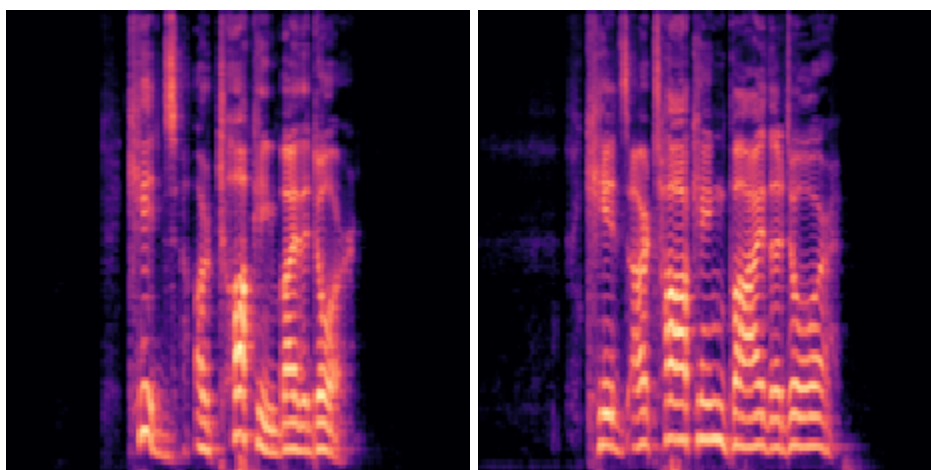


Figure 1 (left): Example spectrogram of a song clip (angry emotion).

Figure 2 (right): Example spectrogram of a speech clip (angry emotion).

CNN Models Development

We developed and experimented with several CNN architectures to classify spectrogram images. We initially trained models on the combined dataset, but later trained separate models for song and speech spectrograms after observing performance issues.

Combined Spectrograms (Song and Speech)

We started by building a model based on the architecture from Homework 3, removing early stopping and dropout to test their individual effects. We experimented with different combinations of convolutional layers, max pooling layers, dropout layers, and dense layers, and varied their order within the network. Despite these modifications, models exhibited persistent overfitting, with validation accuracy remaining around 70%.

Best Combined Model:

This model architecture achieved the highest validation performance during training and was selected for evaluation in the Results section.

```
base_model = keras.Sequential([
    keras.Input(shape=input_shape),
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu'),
    layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(8, activation='softmax')
])

base_model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
base_model.fit(train_generator, epochs=15, validation_data=val_generator)
```

From this experiment, we observed that combining song and speech spectrograms in a single model led to significant overfitting and limited model generalization. As a result, we decided to separate the datasets and train different models for song and speech clips independently.

Song Spectrograms

After separating the datasets, we repeated model development for song clips. We tested different architectures and adjusted the learning rate from 0.01 to 0.0001, but learning rate changes did not significantly improve performance. We also repeated the experiments we have done in testing the

combined spectrograms, including experimenting with adding dropout layers and using different dropout rates, and using different optimization methods. During this process, we adjust our model by looking at training accuracy and validation accuracy, and loss to determine what is the best way to reduce overfitting and increase our model's performance.

Best Song Model:

This model architecture achieved the highest validation performance during training and was selected for evaluation in the Results section.

```
model4 = keras.Sequential([
    keras.Input(shape=input_shape),
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu'),
    layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D(pool_size=(2, 2)), # add another max pooling
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Flatten(),
    layers.Dropout(0.5),
    layers.Dense(128, activation='relu'),
    layers.Dense(8, activation='softmax'),
])

model4.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])

import keras.callbacks
callbacks = [
    keras.callbacks.EarlyStopping(monitor='val_loss', patience=3) # Quit after 3 rounds of no validation loss improvement
]

model4.fit(train_generator, epochs=15, validation_data=val_generator, callbacks=callbacks) # add early stopping
```

Speech Spectrograms

Similarly, we built CNN models for speech clips. Despite experimenting with different architectures and learning rates, the models showed limited validation improvement.

Best Speech Model:

This model architecture achieved the highest validation performance during training and was selected for evaluation in the Results section.

```

from keras import regularizers
from keras import layers, regularizers, Sequential

model6 = Sequential(
    [
        keras.Input(shape=input_shape),
        layers.Conv2D(32, kernel_size=(3, 3), activation='relu', kernel_regularizer=regularizers.l2(1e-4)), # add l2 regularization
        # remove max pooling
        layers.Conv2D(64, kernel_size=(3, 3), activation='relu', kernel_regularizer=regularizers.l2(1e-4)), #add l2 regularization
        layers.MaxPooling2D(pool_size=(2, 2)),

        layers.Conv2D(128, (3, 3), activation='relu', kernel_regularizer=regularizers.l2(1e-4)), #add l2 regularization
        # remove max pooling
        layers.Conv2D(256, (3, 3), activation='relu', kernel_regularizer=regularizers.l2(1e-4)), #add l2 regularization
        layers.MaxPooling2D(pool_size=(2, 2)),

        layers.Flatten(),
        layers.Dropout(0.6), # change dropout from 0.5 to 0.6
        layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(1e-4)), #add l2 regularization
        layers.Dense(8, activation='softmax'),
    ]
)

model6.compile(loss="categorical_crossentropy", optimizer=keras.optimizers.Adam(learning_rate=1e-4), metrics=["accuracy"]) # modify learning rate

import keras.callbacks
callbacks = [
    keras.callbacks.EarlyStopping(monitor='val_loss', patience=3) # Quit after 3 rounds of no validation loss improvement
]

model6.fit(train_generator, epochs=15, validation_data=val_generator, callbacks=callbacks)

```

Data Augmentation

Due to the observed overfitting in our CNN models, we hypothesized that the models were struggling to generalize because of the limited diversity and quantity of training samples. To address this issue and improve model generalization, we applied several data augmentation techniques to the spectrograms, including adding noise, altering pitch, and adjusting speed.

The augmentation parameters we explored were as follows:

- Noise Injection: noise levels of 0.005, 0.01, 0.02, 0.05, 0.1, and 0.2
- Pitch Shifting: shifts of -4, -2, +2, and +4 semitones
- Speed Modification: speed factors of 0.25, 0.4, 0.5, 0.8, 1.2, 1.5, 2.0, and 2.5

For each augmentation method, we first evaluated each augmentation method individually and selected the best-performing parameter, defined as the one whose model achieved the highest validation accuracy with less than a 5% drop from the original song model. Using these parameters, we generated three augmented versions of the original dataset by randomly applying combinations of noise, pitch, and speed modifications. These were combined with the original spectrograms to create a more diverse training set used to improve model robustness.

Training and Evaluation

After generating the augmented spectrograms, we evaluated the validation performance for each augmentation parameter. Based on these results, we selected the augmented spectrograms with the highest validation accuracy and merged them with the original spectrograms to form an expanded training dataset. We then retrained selected CNN models using this combined dataset to evaluate whether data augmentation improved model generalization and validation accuracy.

Fine-Tuning Experiment

To improve the performance and generalization of our CNN models, we conducted a series of fine-tuning experiments. The experiments focused on adjusting the hyperparameters such as batch size, learning rate, regularization strategies, and model depth.

1. Batch Size

We evaluated the effect of batch size on training stability and convergence speed. Initially, a batch size of 32 was used, but we later reduced it to 16 to allow for better generalization on the validation set.

2. Learning Rate

The learning rate was a key focus of tuning. We tested learning rates of $1e-3$, $5e-4$, and $1e-4$. The final model was trained with $1e-4$, which yielded the most stable learning dynamics and lowest validation loss without premature convergence.

3. Regularization

To mitigate overfitting, L2 regularization was applied to all convolutional layers. We tested the regularization parameter of $l_2 = 1e-4$, $1e-5$, and $1e-6$. The final model adopts $l_2 = 1e-4$, which penalizes large weights and improves model generalization with the best overall performance.

In the fully connected layer, a combined L1 and L2 regularization ($l_1=1e-5$, $l_2=1e-4$) was introduced to encourage sparsity and prevent co-adaptation. Dropout layers with a dropout rate of 0.5 were added after both the flattening layer and the first dense layer to further reduce overfitting.

5. Early Stopping

To prevent overfitting and reduce training time, early stopping was employed. We tested the patience of 3, 4, and 5 epochs, and we finally found that 3 epochs is most reasonable for early stopping. This stopped training when the validation loss did not improve for 3 consecutive epochs.

Transfer Learning

After training our initial CNN models, we performed transfer learning to investigate whether using pretrained models could further improve emotion classification performance. For this, we used a pretrained CNN model, ResNet50, which was originally trained on the ImageNet dataset. We replaced the top classification layers with new dense layers suited for our 8 emotion classes. We initially froze the pretrained convolutional base to retain learned feature representations and trained only the newly added layers. Through transfer learning, we aimed to leverage the general visual pattern recognition capabilities of large CNNs to enhance spectrogram-based emotion recognition, especially in our limited-data setting.

After training, the model was evaluated on an unseen validation and real-world test dataset to assess its generalization ability.

Results

Raw Dataset CNN Models Evaluation

The best combined model achieved a training accuracy of 97.85%, a training loss of 0.0811, a validation accuracy of 64.96%, and a validation loss of 1.7480. Although the model fit the training data well, the large gap between training and validation performance indicates significant overfitting, suggesting that combining song and speech spectrograms limited the generalization ability.

The best song model achieved a training accuracy of 98.05%, a training loss of 0.0584, a validation accuracy of 89.39%, and a validation loss of 0.3066. Compared to the combined

model, the song-only model showed much better generalization, with validation accuracy much closer to training accuracy and a substantially lower validation loss.

The best speech model achieved a training accuracy of 95.00%, a training loss of 0.2479, a validation accuracy of 59.30%, and a validation loss of 1.3842. We believe the lower validation performance on speech spectrograms is due to higher variability and less consistent frequency patterns compared to song spectrograms.

Data Augmentation Results

Noise Injection Spectrograms Validation Accuracy

Parameters	Song Spectrograms	$\Delta\%$ (vs. original)	Speech Spectrograms	$\Delta\%$ (vs. original)
0.005	86.36%	-3.39%	49.47%	-16.55%
0.01	87.88%	-1.69%	42.81%	-27.77%
0.02	83.33%	-6.80%	41.40%	-30.14%
0.05	76.77%	-14.12%	29.47%	-50.23%
0.1	70.20%	-21.47%	25.96%	-56.14%
0.2	60.10%	-32.77%	28.42%	-52.00%

Pitch Shifting Spectrograms Validation Accuracy

Parameters	Song Spectrograms	$\Delta\%$ (vs. original)	Speech Spectrograms	$\Delta\%$ (vs. original)
-4	82.32%	-7.91%	62.11%	4.73%
-2	86.36%	-3.39%	61.40%	3.54%
+2	87.37%	-2.26%	57.19%	-3.55%

+4	83.84%	-6.21%	60.00%	1.18%
----	--------	--------	--------	-------

Speed Modification Spectrograms Validation Accuracy

Parameters	Song Spectrograms	$\Delta\%$ (vs. original)	Speech Spectrograms	$\Delta\%$ (vs. original)
0.25	83.84%	-6.21%	58.25%	-1.77%
0.4	87.88%	-1.69%	56.49%	-4.74%
0.5	85.86%	-3.95%	63.16%	6.51%
0.8	86.36%	-3.39%	59.65%	0.59%
1.2	87.88%	-1.69%	61.05%	3.42%
1.5	85.86%	-3.95%	61.75%	4.13%
2.0	85.35%	-4.52%	56.49%	-4.74%
2.5	86.87%	-2.82%	58.95%	-0.59%

For song spectrograms, a noise level of 0.005 and 0.01, pitch shifting by -2 or +2 semitones, and a speed factor of 0.4, 0.5, 0.8, 1.2, 1.5, 2.0, and 2.5 resulted in less than 5% validation accuracy drop compared to the original model.

For speech spectrograms, pitch shifting by -4, -2, +2, and +4 semitones, and a speed factor of 0.25, 0.4, 0.5, 0.8, 1.2, 1.5, 2.0, and 2.5 resulted in less than 5% validation accuracy drop compared to the original model.

Among the three augmentation techniques, noise injection caused the largest drop in validation accuracy, especially for speech spectrograms. This may be because adding noise disrupts important temporal and spectral features that the model relies on to detect emotions. On the other hand, pitch shifting and speed modification had a smaller impact, usually changing accuracy by

less than 5 percent, since these methods maintain the overall structure of the spectrogram and allow the model to recognize key patterns.

Final Models Performance

Song Spectrograms

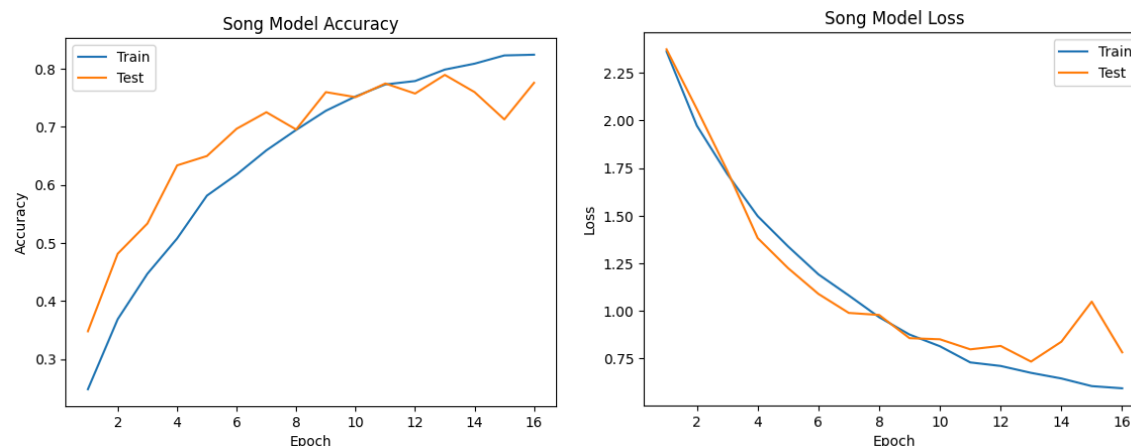
```
model4 = keras.Sequential([
    keras.Input(shape=input_shape),
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu', kernel_regularizer=regularizers.l2(1e-4)),
    layers.BatchNormalization(),
    layers.Conv2D(64, kernel_size=(3, 3), activation='relu', kernel_regularizer=regularizers.l2(1e-4)),
    layers.MaxPooling2D(pool_size=(2, 2)), # add another max pooling
    layers.Conv2D(128, (3, 3), activation='relu', kernel_regularizer=regularizers.l2(1e-4)),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Flatten(),
    layers.Dropout(0.5),
    layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l1_l2(l1=1e-5, l2=1e-4)),
    layers.Dropout(0.5),
    layers.Dense(8, activation='softmax'),
])

model4.compile(loss="categorical_crossentropy", optimizer=Adam(1e-4), metrics=["accuracy"])

import keras.callbacks
callbacks = [
    keras.callbacks.EarlyStopping(monitor='val_loss', patience=3) # Quit after 3 rounds of no validation loss improvement
]

model4.fit(train_generator, epochs=30, validation_data=val_generator, callbacks=callbacks) # add early stopping
```

The final song model achieved a training accuracy of 82.29% and a validation accuracy of 77.60%, with a training loss of 0.6120 and a validation loss of 0.7832. Compared to the earlier song CNN model, the final model has slightly lower validation accuracy. However, due to data augmentation and architecture refinements, it no longer overfits and was trained on a more diverse dataset. This suggests better generalization, making the slight drop in accuracy acceptable.



The model's overall weighted F1 score is 0.77. The model performed especially well in classifying neutral (F1: 0.97), calm (F1: 0.85), happy (F1: 0.81), and angry (F1: 0.81) emotions. However, performance was poor on fearful (F1: 0.57) and failed entirely on disgust and surprised, likely due to a lack of representative samples.

	precision	recall	f1-score	support
angry	0.91	0.73	0.81	147
calm	0.80	0.90	0.85	147
disgust	0.00	0.00	0.00	0
fearful	0.73	0.46	0.57	147
happy	0.80	0.82	0.81	147
neutral	0.99	0.96	0.97	73
sad	0.61	0.88	0.72	147
surprised	0.00	0.00	0.00	0
accuracy			0.78	808
macro avg	0.61	0.59	0.59	808
weighted avg	0.79	0.78	0.77	808

Speech Spectrograms

```
speech_model = Sequential(
    [
        keras.Input(shape=input_shape),
        layers.Conv2D(32, kernel_size=(3, 3), activation='relu', kernel_regularizer=regularizers.l2(1e-4)), # add l2 regularization
        # remove max pooling
        layers.Conv2D(64, kernel_size=(3, 3), activation='relu', kernel_regularizer=regularizers.l2(1e-4)), #add l2 regularization
        layers.MaxPooling2D(pool_size=(2, 2)),

        layers.Conv2D(128, (3, 3), activation='relu', kernel_regularizer=regularizers.l2(1e-4)), #add l2 regularization
        # remove max pooling
        layers.Conv2D(256, (3, 3), activation='relu', kernel_regularizer=regularizers.l2(1e-4)), #add l2 regularization
        layers.MaxPooling2D(pool_size=(2, 2)),

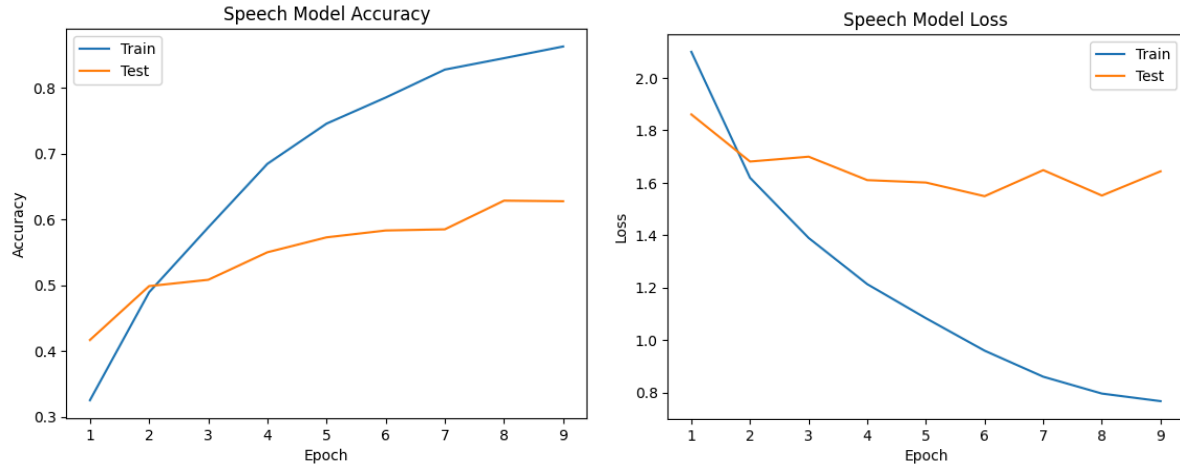
        layers.Flatten(),
        layers.Dropout(0.5),
        layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l1_l2(l1=1e-5, l2=1e-4)),
        layers.Dropout(0.5),
        layers.Dense(8, activation='softmax'),
    ]
)

speech_model.compile(loss="categorical_crossentropy", optimizer=keras.optimizers.Adam(learning_rate=1e-4), metrics=["accuracy"]) # modify learning rate

import keras.callbacks
callbacks = [
    keras.callbacks.EarlyStopping(monitor='val_loss', patience=3) # Quit after 3 rounds of no validation loss improvement
]

history = speech_model.fit(train_generator, epochs=15, validation_data=val_generator, callbacks=callbacks)
```

The final speech model achieved a training accuracy of 85.78% and a validation accuracy of 62.77%, with a training loss of 0.7788 and a validation loss of 1.6437. Compared to the earlier speech CNN model, the final model has higher validation accuracy. This improvement suggests that the final model generalizes better to unseen speech data, likely due to refined architecture and data preprocessing.



The model's overall weighted F1 score is 0.63. The model performed relatively well in classifying disgust (F1: 0.72), calm (F1: 0.71), and angry (F1: 0.68). However, performance was moderate on neutral (F1: 0.64) and fearful (F1: 0.63), and lower for happy (F1: 0.57), surprised (F1: 0.61), and sad (F1: 0.49). This variation in performance suggests that some emotions, particularly sad and happy, are harder to distinguish from others in speech, possibly due to subtle acoustic differences or overlapping vocal expressions.

	precision	recall	f1-score	support
angry	1.00	0.52	0.68	153
calm	0.66	0.75	0.71	153
disgust	0.66	0.78	0.72	153
fearful	0.88	0.49	0.63	153
happy	0.70	0.48	0.57	153
neutral	0.56	0.74	0.64	76
sad	0.41	0.61	0.49	153
surprised	0.54	0.71	0.61	153
accuracy			0.63	1147
macro avg	0.68	0.63	0.63	1147
weighted avg	0.69	0.63	0.63	1147

Comparison between Song and Speech

The song model consistently outperformed the speech model in both validation accuracy and generalization. This discrepancy can be attributed to key differences in the acoustic and structural properties of the data. Song clips tend to have more consistent rhythm, pitch, and harmonic patterns, which translate into clearer and more distinguishable spectrogram features for CNNs to learn from. In contrast, speech clips are more variable, influenced by differences in speaker pace, intonation, and articulation, which introduce noise and irregularity in the spectrograms.

Transfer Learning Results

Using transfer learning with ResNet50, the song model achieved a validation accuracy of 37.75%, which is lower than our final model without transfer learning. This result suggests that features learned from natural images in ImageNet do not transfer well to spectrograms, highlighting the importance of domain-specific architectures in audio-based emotion recognition. While we found models like YAMNet and PANNs are designed for audio and show promise, they require audio-based inputs rather than images we have, making direct integration into our current pipeline difficult. In future work, we plan to adapt our input format to support these models and explore hybrid architectures that combine their pretrained embeddings with our CNN. This shift toward audio-specific transfer learning will be a key focus of our continued research.

Real World Testing

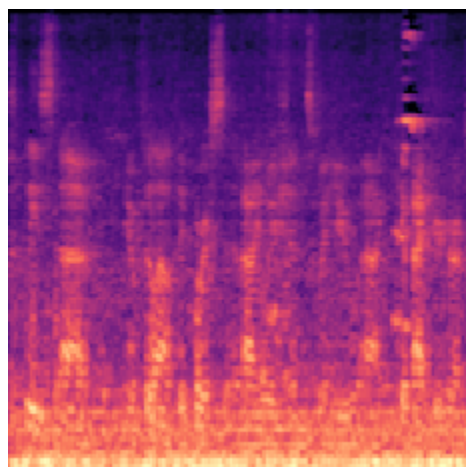


Figure 3. Spectrogram of lecture recording

After generating a spectrogram from a lecture recording where the professor said, “please fill out the course evaluation when you get back to your dorm” we fed it into our trained model. The predicted emotion category was calm, which aligns with our expectation based on the tone of the statement.

Conclusions

In this project, we explored whether convolutional neural networks could effectively classify emotions based on audio spectrograms. By separating song and speech data, we observed

improved model generalization, especially for song spectrograms. Data augmentation helped mitigate overfitting, with pitch shifting and speed modification preserving the integrity of spectrogram structures and resulting in minimal accuracy drops, typically under 5 percent. In contrast, noise injection significantly reduced performance, particularly on speech data, likely due to distortion of key temporal and spectral features essential for emotion recognition.

Our final models achieved a validation accuracy of 77.60% for songs and 62.77% for speech, demonstrating the potential of CNNs in audio-based emotion recognition. The song model performed especially well in classifying neutral, calm, happy, and angry emotions, while the speech model showed higher accuracy for disgust, calm, and angry categories. These results highlight the importance of carefully structuring the dataset and applying augmentation techniques to improve generalization. While accuracy varied across modalities, the final models reflect a balance between performance and robustness. While experimenting with transfer learning, we found that using a model pre-trained on ImageNet resulted in poor performance. This is likely because ImageNet consists of natural images, which differ significantly from spectrograms. As a result, accuracy was lower than that of our custom-built CNN models. In the future, we may involve using audio-specific pretrained models or exploring attention mechanisms to enhance recognition accuracy, especially for more variable speech inputs.

In the future, we could explore audio-specific pretrained models like YAMNet and incorporate attention mechanisms to better capture temporal dependencies in speech data. Additionally, expanding the dataset to include more diverse real-world recordings and exploring multi-modal approaches that combine audio and text could further improve performance and real-world applicability.

References

- OpenAI. ChatGPT. Version 4o, 2025, <https://openai.com/chatgpt>. Used for code organization.
- Sreehari R. "How to Convert A .WAV File to a Spectrogram in Python3." Stack Overflow, June 27, 2017, stackoverflow.com/questions/44787437/how-to-convert-a-wav-file-to-a-spectrogram-in-python3. Accessed 30 Apr. 2025.